SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE FAKULTA ELEKTROTECHNIKY A INFORMATIKY

KONVOLUČNÁ NEURÓNOVÁ SIEŤ

ZADANIE 2

2024

Matej Krumlovský

Obsah

1. Zadanie úlohy
2. Postup riešenia
   - 2.1. Dataset
   - 2.2. Trénovanie CNN
   - 2.3. Obrázok vizualizácie siete
   - 2.4. Graf priebehu siete
   - 2.5. Testovanie obrázku z každej triedy a predikcia
   - 2.6. Predikcia všetkých obrázkov v cykle
   - 2.7. Skóre presnosti, konfúzna matica, klasifikačný report
   - 2.8. Uloženie, načítanie a skompilovanie siete
   - 2.9. Experimenty
     - 2.9.1. Experiment 1
     - 2.9.2. Experiment 2
     - 2.9.3. Experiment 3
   - 2.10. Vyhodnotenie experimentov
     - 2.10.1. Experiment 1
     - 2.10.2. Experiment 2
     - 2.10.3. Experiment 3
   - 2.11. Záver

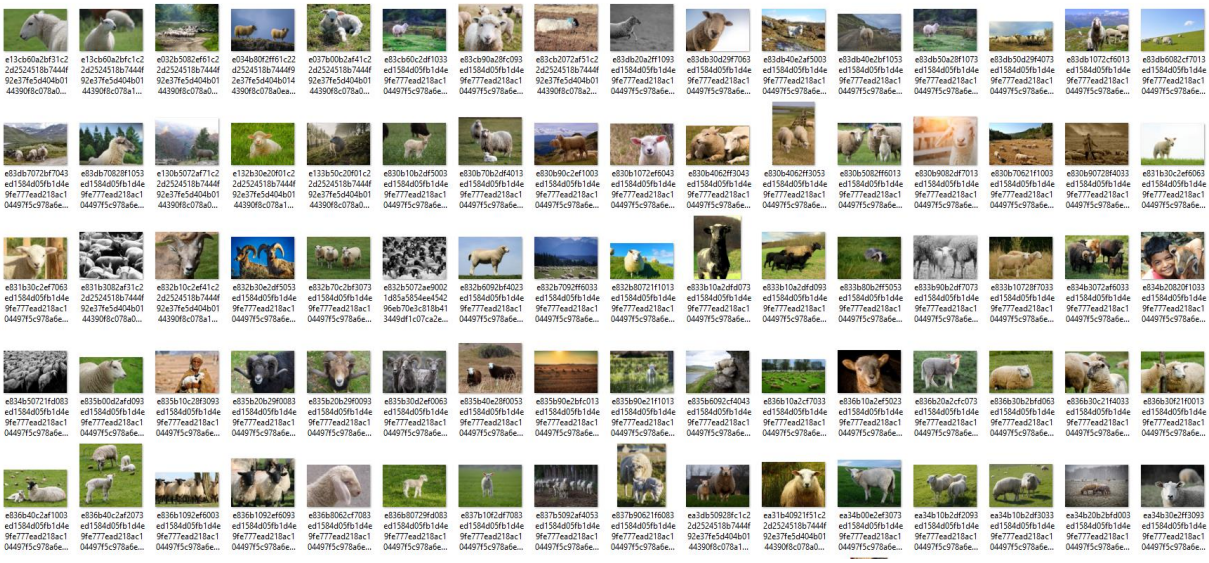
1 Zadanie úlohy

Našou úlohou je vytvoriť konvolučnú neurónovú sieť na rozpoznávanie 4 a viac kategórií obrázkov. Testovanie máme vykonávať na viacerých druhov sietí a následne vyhodnotiť ich úspešnosť.

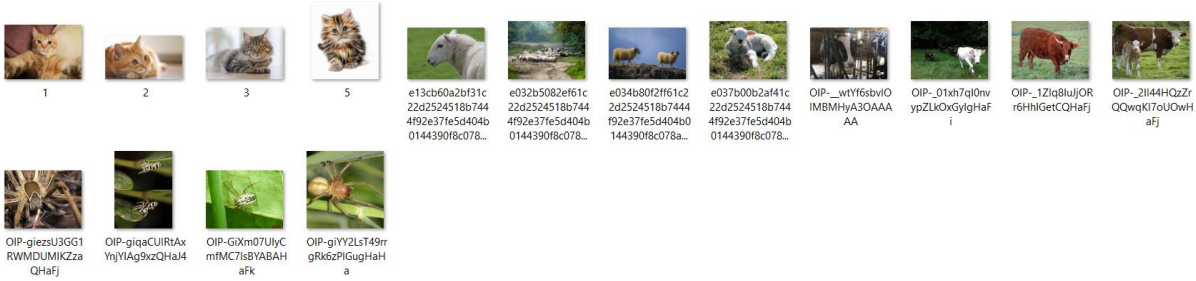
2 Postup riešenia

2.1 Dataset

Trénovanie bolo na 4 kategóriach pričom každá obsahuje 1800 obrázkov. Pri testovaní každá kategória obsahuje 310 obrázkov. Názornú ukážku obrázkov môžeme vidieť na obrázku 2.



Obr. 1: Ukážka datasetu – kategória ovca



Obr. 2: Ukážka obrázku každej triedy

Na preprocesovanie datasetu a využijeme tf.keras.preprocessing.image Obrázky zmenšíme na veľkosť 64*64, classmode nastavýme na categorical a batch size na 32. Výsledný kód možeme vidieť nižšie.

```
train_datagen = ImageDataGenerator(rescale=1./255,
                                   shear_range=0.2,
                                   zoom_range=0.2,
                                   horizontal_flip=True)
training_set = train_datagen.flow_from_directory('datasetZ2/TRAINING',
                                                 target_size=(64, 64),
                                                 batch_size=32,
                                                 class_mode='categorical',
                                                 shuffle=True)

test_datagen = ImageDataGenerator(rescale=1./255)
test_set = test_datagen.flow_from_directory('datasetZ2/TEST',
                                            target_size=(64, 64),
                                            batch_size=32,
                                            class_mode='categorical',
                                            shuffle=False)
```

```
Found 7187 images belonging to 4 classes.
Found 1249 images belonging to 4 classes.
```

2.2 Trénovanie CNN

Na trénovanie využijeme neurónovú sieť, jej štruktúru môžeme vidieť nižšie

```
cnn = tf.keras.models.Sequential()
cnn.add(tf.keras.layers.Conv2D(filters=32, kernel_size=3, activation='relu', input_shape=[64, 64, 3]))
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
cnn.add(tf.keras.layers.Conv2D(filters=64, kernel_size=3, activation='relu'))
cnn.add(tf.keras.layers.Flatten())
cnn.add(tf.keras.layers.Dense(units=128, activation='relu'))
cnn.add(tf.keras.layers.Dense(units=4, activation='softmax'))  # 4 triedy
```

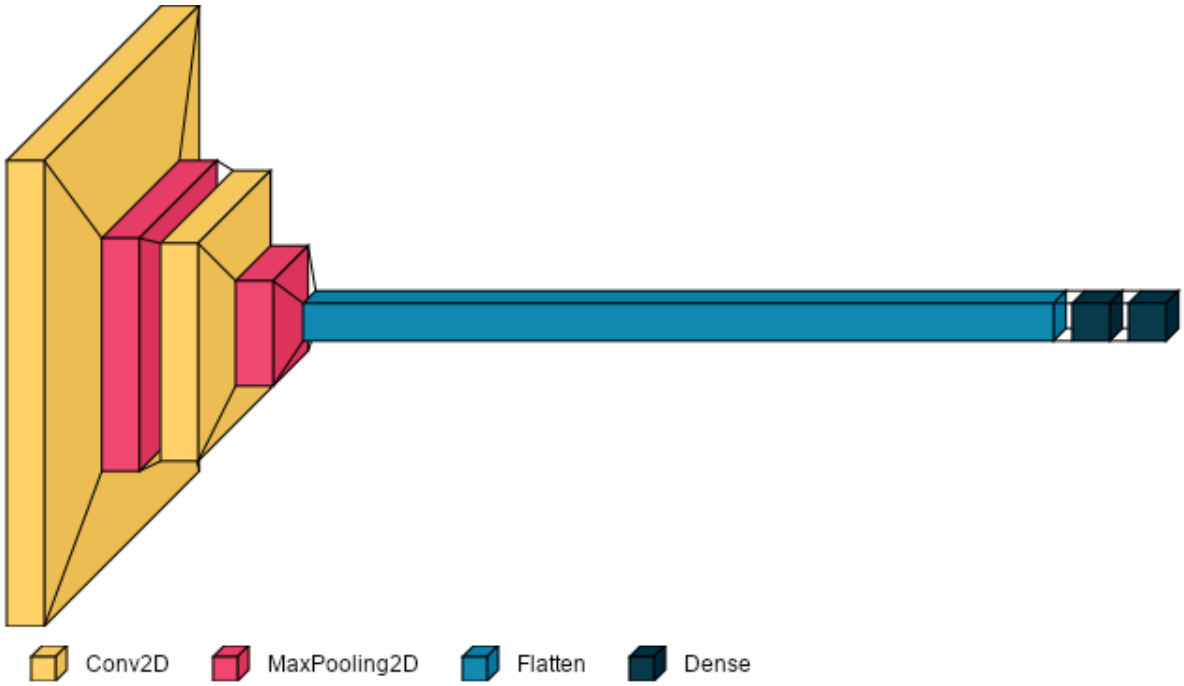
Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_5 (Conv2D)	(None, 29, 29, 32)	9,248
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 32)	0
flatten_2 (Flatten)	(None, 6272)	0
dense_4 (Dense)	(None, 128)	802,944
dense_5 (Dense)	(None, 4)	516

Obr. 3: Sieť 1 – architektúra

```
Class indices: {'krava': 0, 'macka': 1, 'ovca': 2, 'pavuk': 3}
```

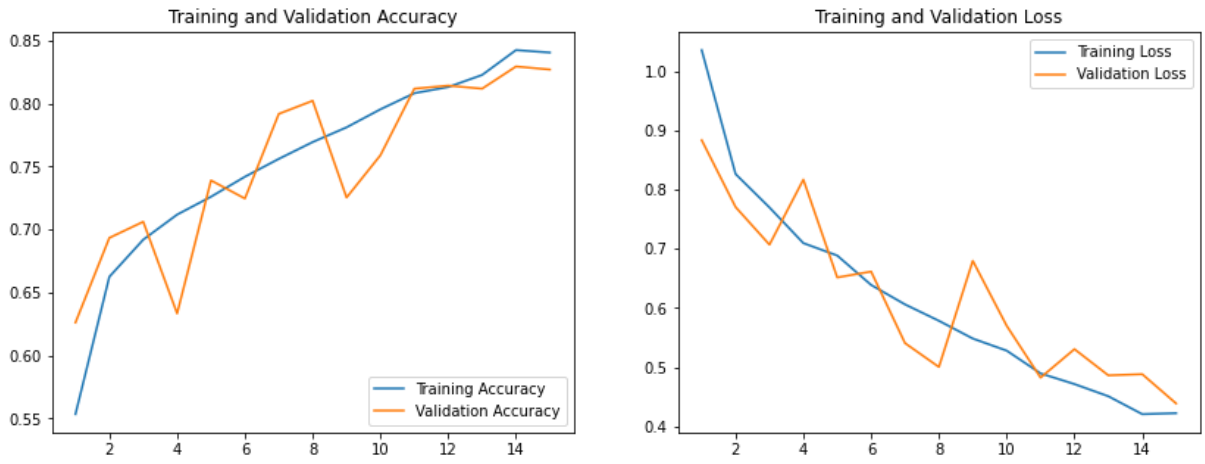
Obr. 4: Klasifikačné triedy

2.3 Obrázok vizualizácie siete



Obr. 5: Vizualizácia siete

2.4 Graf priebehu siete



Obr. 6: Grafy priebehu trénovania

2.5 Testovanie obrázku z každej triedy a predikcia

Obr. 7: Krava

```
1/1 [==============================] - 0s 133ms/step
Výstup modelu pre daný obrázok ovca: [[0.29258323 0.22225632 0.23175289 0.25340754]]
Na obrázku je: krava
```

Obr. 8: Macka

```
1/1 [==============================] - 0s 130ms/step
Výstup modelu pre daný obrázok ovca: [[0.24992429 0.27072242 0.23424247 0.24511081]]
Na obrázku je: macka
```

Obr. 9: Ovca

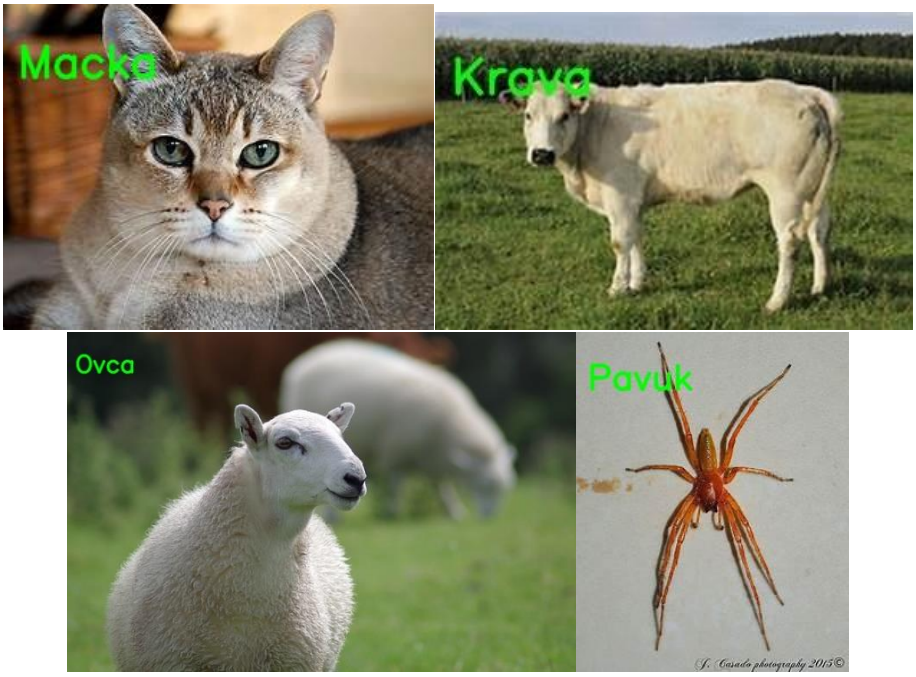
```
1/1 [==============================] - 0s 136ms/step
Výstup modelu pre daný obrázok ovca: [[0.23877962 0.21975513 0.28761375 0.25385147]]
Na obrázku je: ovca
```

Obr. 10: Pavúk

```
1/1 [==============================] - 0s 128ms/step
Výstup modelu pre daný obrázok ovca: [[0.23847146 0.24921216 0.23976561 0.27255082]]
Na obrázku je: pavuk
```

2.6 Predikcia všetkých obrázkov v cykle

Úspešnosť môžeme pozorovať z obrázkov. Vidíme, že sieť klasifikovala správne všetky 4 obrázky



Obr. 11: Predikované obrázky s textom

Výsledky vypísané v konzole môžeme vidieť nižšie

```
Spracováva sa: kravicka.jpeg
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step
[[6.4551175e-01 6.5188491e-05 3.5442287e-01 1.3635298e-07]]
Na obrázku je: krava

Spracováva sa: macka.jpeg
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
[[0.00083378 0.8143571  0.01285388 0.17195521]]
Na obrázku je: macka

Spracováva sa: ovecka.jpg
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
[[0.11162426 0.15067184 0.72761357 0.01009027]]
Na obrázku je: ovca

Spracováva sa: pavuk.jpeg
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
[[2.5745470e-05 8.8615234e-05 1.7886674e-04 9.9970669e-01]]
Na obrázku je: pavuk
```

Použili sme kód ktorý sa skladá z troch funkcií, ktoré riešia tieto úlohy. Pre túto poslednú úlohu som použil funkciu showPrint() ktorá vloží text do obrázkov pre každú triedu. Výsledný kód tejto funkcie je uvedený nižšie.

Zadefinovanie funkcie

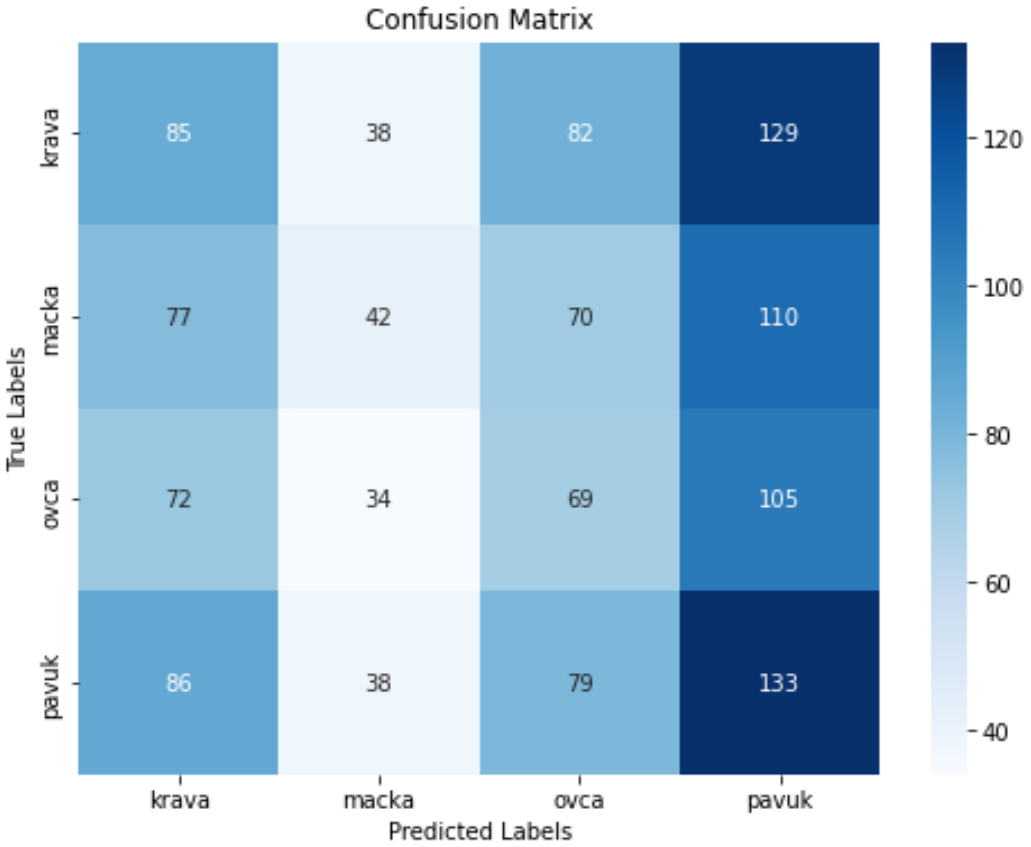
```
def showPrint(name, predikcia, slovnik, single_name):
    im = cv2.imread(name)
    text = slovnik[predikcia[0]]
    cv2.putText(im, text, (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)
    cv2.imwrite(f"predicted_{single_name}.jpg", im)
```

Volanie funkcie

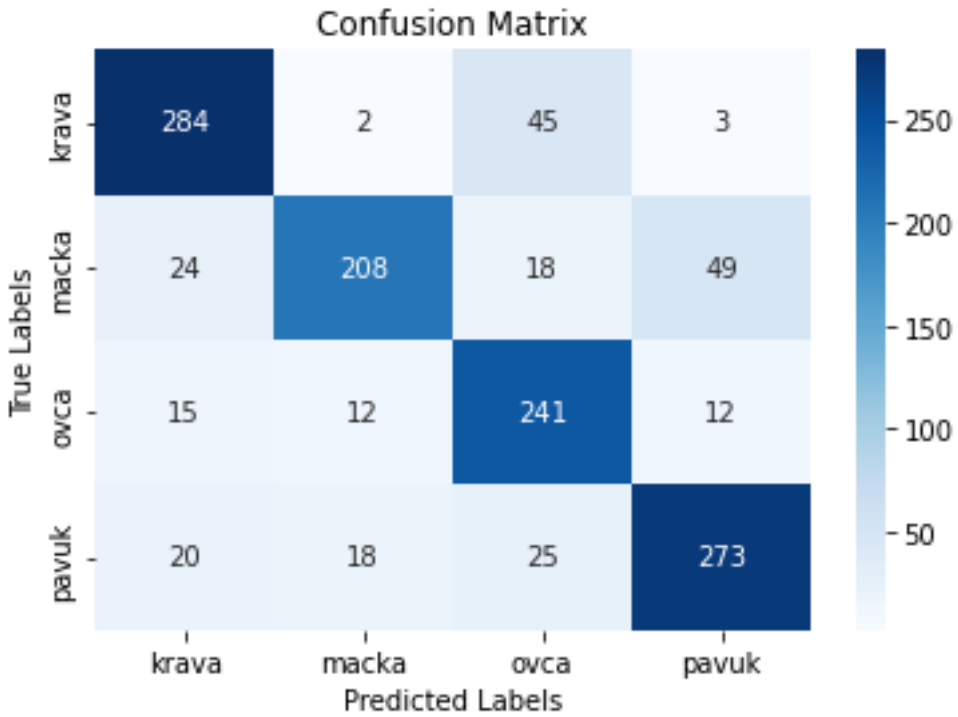
```
location = ""
images = ["kravicka.jpeg", "macka.jpeg", "ovecka.jpg", "pavuk.jpeg"]
print(location)
slovnik = {0: "Krava", 1: "Macka", 2: "Ovca", 3: "Pavuk"}
for i in images:
    print(f"\nSpracováva sa: {i}")
    name = location + i
    result = oneImageTest(name)
    predict = predictedClass(result)
    showPrint(name, predict, slovnik, i)
```

2.7 Skóre presnosti, konfúzna matica, klasifikačný report

Presnosť matice cez accuracy_score() dosahuje . Klasifikačná matica nám ukazuje koľko výsledkov máme správnych podľa našich tried 1-4. Našou snahou je dosiahnuť čo najväčšie čísla na diagonále. Ako môžeme vidieť naša sieť si pomerne často zamieňala zvieratá, presnosť je niečo malo cez 20%, ale je to preto lebo hodnota epochs je nastavená na číslo 7, teraz počet epochs nastavím na hodnotu 15 a presnosť by mala vyť výrazne vyššia a bude to aj vidno na druhej konfúznej matici, kde sa na diagonále budú nachádzať najväčšie čísla.



Obr. 12: Konfúzna matica pre epochs=7



Obr. 13: Konfúzna matica pre epochs=15

```
Accuracy score: 0.8054
Confusion Matrix:
[[284   2  45   3]
 [ 24 208  18  49]
 [ 15  12 241  12]
 [ 20  18  25 273]]
Classification Report:
              precision    recall  f1-score   support

       krava       0.83      0.85      0.84       334
       macka       0.87      0.70      0.77       299
        ovca       0.73      0.86      0.79       280
       pavuk       0.81      0.81      0.81       336

    accuracy                           0.81      1249
   macro avg       0.81      0.80      0.80      1249
weighted avg       0.81      0.81      0.80      1249
```

Obr. 14: Presnosť, konfúzna matica a klasifikačný report v konzole

2.8 Uloženie, načítanie a skompilovanie siete

Uskutočníme to pomocou kódu

```python
cnn.save('cnn_model_v1.h5')  # Uloženie váh modelu
model_json = cnn.to_json()  # Exportovanie architektúry modelu
with open('cnn_model_v1.json', 'w') as json_file:
    json_file.write(model_json)

# Načítanie uloženého modelu
with open('cnn_model_v1.json', 'r') as json_file:
    json_saved_model = json_file.read()

# Výpis JSON architektúry
print("Architektúra modelu v JSON formáte:")
print(json_saved_model)

network_loaded = tf.keras.models.model_from_json(json_saved_model)
network_loaded.load_weights('cnn_model_v1.h5')  # Načítanie uložených váh
network_loaded.compile(loss='categorical_crossentropy', optimizer='Adam', metrics=['accuracy'])

# Zobrazenie architektúry načítaného modelu
print("Načítaný model:")
network_loaded.summary()
```

```
Architektúra modelu v JSON formáte:
{"module": "keras", "class_name": "Sequential", "config": {"name": "sequential_6", "trainable": true, "dtype": {"module": "keras", "class_name":
"DTypePolicy", "config": {"name": "float32"}, "registered_name": null}, "layers": [{"module": "keras.layers", "class_name": "InputLayer", "config":
{"batch_shape": [null, 64, 64, 3], "dtype": "float32", "sparse": false, "name": "input_layer_6"}, "registered_name": null}, {"module": "keras.layers",
"class_name": "Conv2D", "config": {"name": "conv2d_12", "trainable": true, "dtype": {"module": "keras", "class_name": "DTypePolicy", "config": {"name":
"float32"}, "registered_name": null}, "filters": 32, "kernel_size": [3, 3], "strides": [1, 1], "padding": "valid", "data_format": "channels_last",
"dilation_rate": [1, 1], "groups": 1, "activation": "relu", "use_bias": true, "kernel_initializer": {"module": "keras.initializers", "class_name":
"GlorotUniform", "config": {"seed": null}, "registered_name": null}, "bias_initializer": {"module": "keras.initializers", "class_name": "Zeros",
"config": {}, "registered_name": null}, "kernel_regularizer": null, "bias_regularizer": null, "activity_regularizer": null, "kernel_constraint": null,
"bias_constraint": null}, "registered_name": null, "build_config": {"input_shape": [null, 64, 64, 3]}}, {"module": "keras.layers", "class_name":
"MaxPooling2D", "config": {"name": "max_pooling2d_6", "trainable": true, "dtype": {"module": "keras", "class_name": "DTypePolicy", "config": {"name":
"float32"}, "registered_name": null}, "pool_size": [2, 2], "padding": "valid", "strides": [2, 2], "data_format": "channels_last"}, "registered_name":
null}, {"module": "keras.layers", "class_name": "Conv2D", "config": {"name": "conv2d_13", "trainable": true, "dtype": {"module": "keras", "class_name":
"DTypePolicy", "config": {"name": "float32"}, "registered_name": null}, "filters": 64, "kernel_size": [3, 3], "strides": [1, 1], "padding": "valid",
"data_format": "channels_last", "dilation_rate": [1, 1], "groups": 1, "activation": "relu", "use_bias": true, "kernel_initializer": {"module":
"keras.initializers", "class_name": "GlorotUniform", "config": {"seed": null}, "registered_name": null}, "bias_initializer": {"module":
"keras.initializers", "class_name": "Zeros", "config": {}, "registered_name": null}, "kernel_regularizer": null, "bias_regularizer": null,
"activity_regularizer": null, "kernel_constraint": null, "bias_constraint": null}, "registered_name": null, "build_config": {"input_shape": [null, 31,
31, 32]}}, {"module": "keras.layers", "class_name": "Flatten", "config": {"name": "flatten_6", "trainable": true, "dtype": {"module": "keras",
"class_name": "DTypePolicy", "config": {"name": "float32"}, "registered_name": null}, "data_format": "channels_last"}, "registered_name": null,
"build_config": {"input_shape": [null, 29, 29, 64]}}, {"module": "keras.layers", "class_name": "Dense", "config": {"name": "dense_12", "trainable":
true, "dtype": {"module": "keras", "class_name": "DTypePolicy", "config": {"name": "float32"}, "registered_name": null}, "units": 128, "activation":
"relu", "use_bias": true, "kernel_initializer": {"module": "keras.initializers", "class_name": "GlorotUniform", "config": {"seed": null},
"registered_name": null}, "bias_initializer": {"module": "keras.initializers", "class_name": "Zeros", "config": {}, "registered_name": null},
"kernel_regularizer": null, "bias_regularizer": null, "kernel_constraint": null, "bias_constraint": null}, "registered_name": null, "build_config":
{"input_shape": [null, 53824]}}, {"module": "keras.layers", "class_name": "Dense", "config": {"name": "dense_13", "trainable": true, "dtype":
{"module": "keras", "class_name": "DTypePolicy", "config": {"name": "float32"}, "registered_name": null}, "units": 4, "activation": "softmax",
"use_bias": true, "kernel_initializer": {"module": "keras.initializers", "class_name": "GlorotUniform", "config": {"seed": null}, "registered_name":
null}, "bias_initializer": {"module": "keras.initializers", "class_name": "Zeros", "config": {}, "registered_name": null}, "kernel_regularizer": null,
"bias_regularizer": null, "kernel_constraint": null, "bias_constraint": null}, "registered_name": null, "build_config": {"input_shape": [null, 128]}}],
"build_input_shape": [null, 64, 64, 3]}, "registered_name": null, "build_config": {"input_shape": [null, 64, 64, 3]}, "compile_config": {"loss":
"categorical_crossentropy", "loss_weights": null, "metrics": ["accuracy"], "weighted_metrics": null, "run_eagerly": false, "steps_per_execution": 1,
"jit_compile": false}}
```

```
Načítaný model:
Model: "sequential_6"
```

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_13 (Conv2D)	(None, 29, 29, 64)	18,496
flatten_6 (Flatten)	(None, 53824)	0
dense_12 (Dense)	(None, 128)	6,889,600
dense_13 (Dense)	(None, 4)	516

```
Total params: 6,909,508 (26.36 MB)
Trainable params: 6,909,508 (26.36 MB)
Non-trainable params: 0 (0.00 B)
```

Obr. 15: Znovu načítanie siete

2.9 Experimenty

Pri experimentoch so sieťami budeme manipulovať s počtom filtrov, ich veľkosťou, počtom neurónov a batch size atď.

2.9.1 Experiment 1

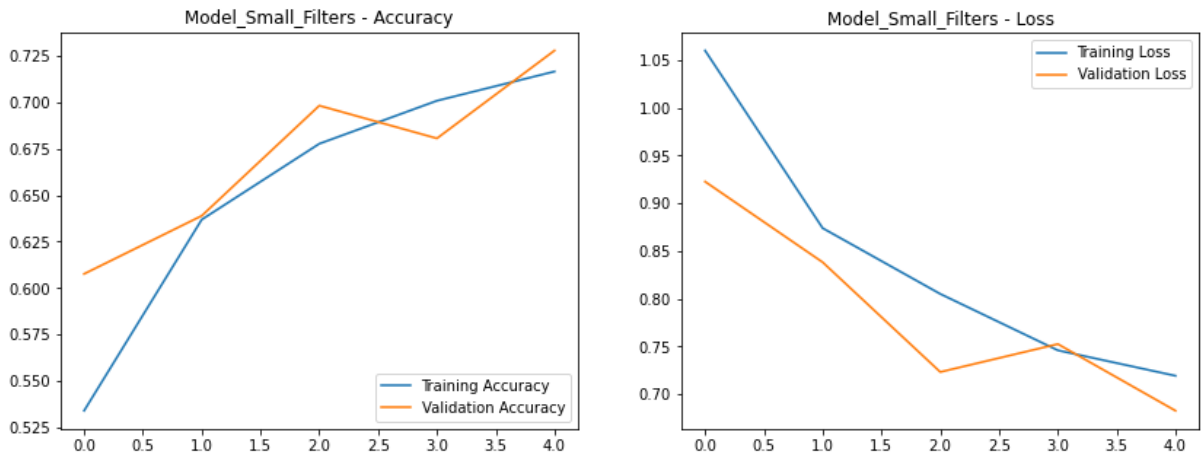
Prvý experiment som nazval model_small_filters, preto lebo používam malé filtre a nízky počet neurónov, počet epochs je nastavený na hodnotu 5.

```
# Experiment 1: Base Model
results.append(build_and_train_model(filters1=32, filters2=64, dense_units=128, epochs=5, model_name="Model_Base"))
```

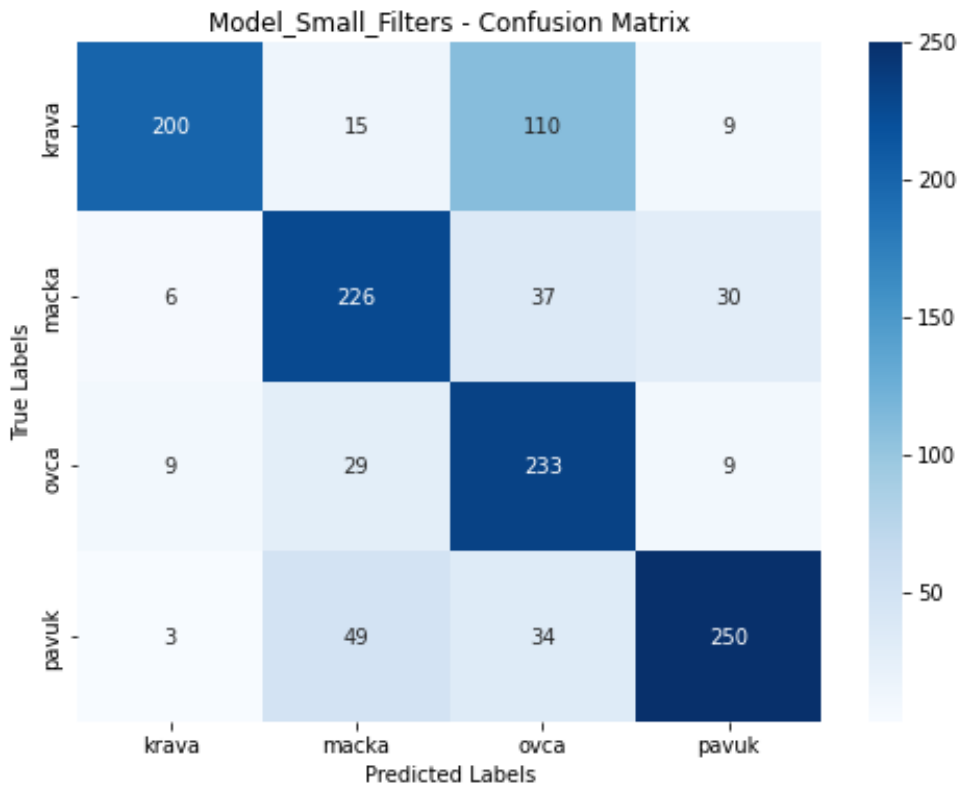
Layer (type)	Output Shape	Param #
conv2d_24 (Conv2D)	(None, 62, 62, 16)	448
max_pooling2d_17 (MaxPooling2D)	(None, 31, 31, 16)	0
conv2d_25 (Conv2D)	(None, 29, 29, 32)	4,640
max_pooling2d_18 (MaxPooling2D)	(None, 14, 14, 32)	0
flatten_12 (Flatten)	(None, 6272)	0
dense_24 (Dense)	(None, 64)	401,472
dense_25 (Dense)	(None, 4)	260

Total params: 406,820 (1.55 MB)
Trainable params: 406,820 (1.55 MB)
Non-trainable params: 0 (0.00 B)

Obr. 16: Sieť 1 – architektúra



Obr. 17: Sieť 1 – Presnosť a strata



Obr. 18: Sieť 1 – Konfúzna matica

```
Model_Small_Filters - Classification Report:
              precision    recall  f1-score   support

       krava       0.92      0.60      0.72       334
       macka       0.71      0.76      0.73       299
        ovca       0.56      0.83      0.67       280
       pavuk       0.84      0.74      0.79       336

    accuracy                           0.73      1249
   macro avg       0.76      0.73      0.73      1249
weighted avg       0.77      0.73      0.73      1249
```

Obr. 19: Sieť 1 – Klasifikačný report

```
Model: Model_Small_Filters
Accuracy: 0.7278
Training Time: 195.11 seconds
Confusion Matrix:
[[200  15 110   9]
 [  6 226  37  30]
 [  9  29 233   9]
 [  3  49  34 250]]
```

Obr. 20: Sieť 1 – Presnosť, čas trénovania a konfúzna matica v konzole

2.9.2 Experiment 2

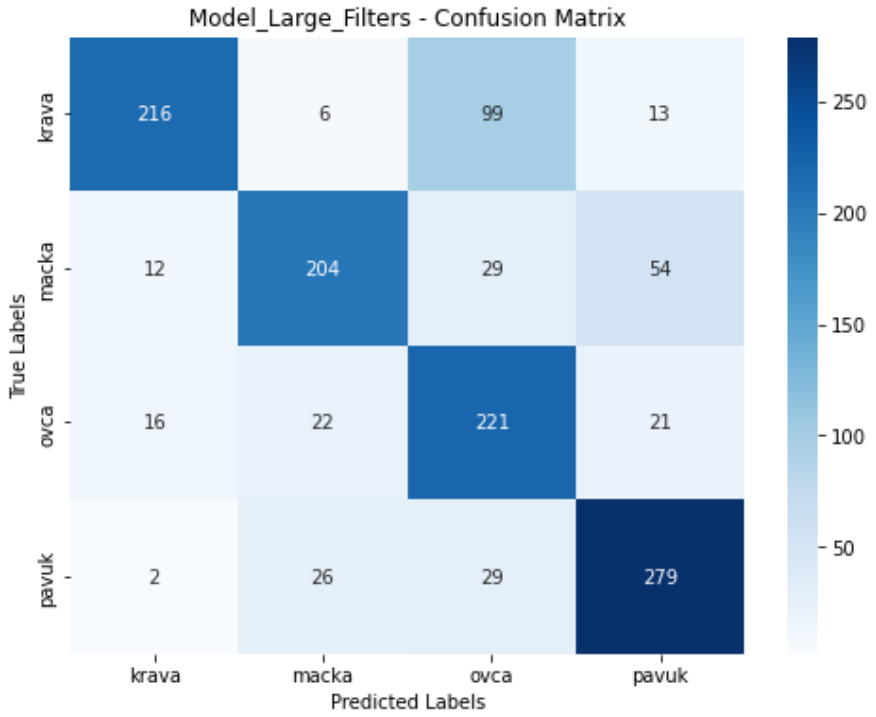
Pri tomto experimente som zväčšil počet a veľkosť filtrov, vrstva obsahuje aj viac neurónov to znamená že poskytuje väčšiu kapacitu modelu, hodnota epochs je 7.

```
# Experiment 2: Enhanced Model
results.append(build_and_train_model(filters1=64, filters2=128, dense_units=256, epochs=7, model_name="Model_Enhanced"))
```

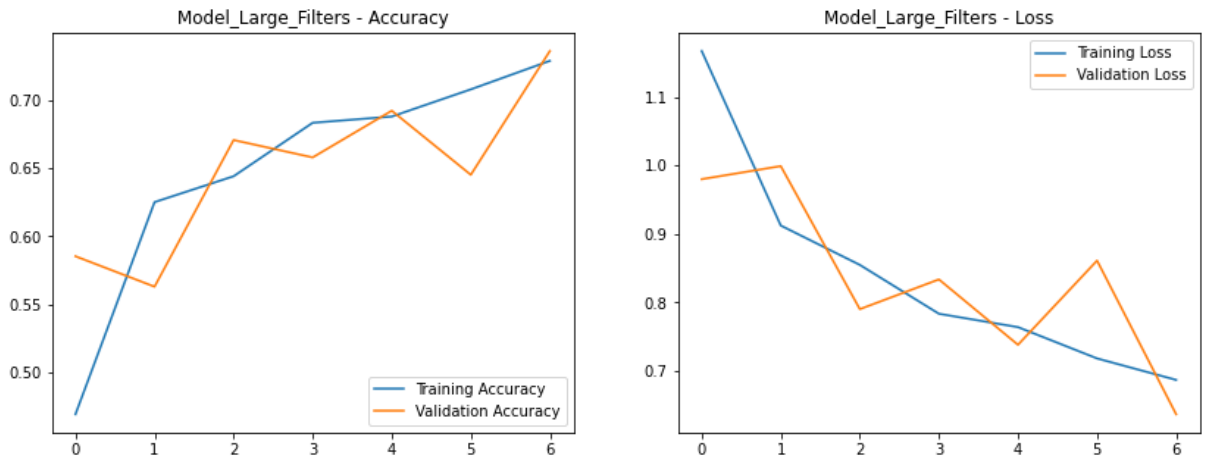
Layer (type)	Output Shape	Param #
conv2d_26 (Conv2D)	(None, 60, 60, 64)	4,864
max_pooling2d_19 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_27 (Conv2D)	(None, 26, 26, 128)	204,928
max_pooling2d_20 (MaxPooling2D)	(None, 13, 13, 128)	0
flatten_13 (Flatten)	(None, 21632)	0
dense_26 (Dense)	(None, 256)	5,538,048
dense_27 (Dense)	(None, 4)	1,028

Total params: 5,748,868 (21.93 MB)
Trainable params: 5,748,868 (21.93 MB)
Non-trainable params: 0 (0.00 B)

Obr. 21: Sieť 2 – Architektúra



Obr. 22: Sieť 2 – Konfúzna matica



Obr. 23: Sieť 2 – Presnosť a strata

```
Model_Large_Filters - Classification Report:
              precision    recall  f1-score   support

       krava       0.88      0.65      0.74       334
       macka       0.79      0.68      0.73       299
        ovca       0.58      0.79      0.67       280
       pavuk       0.76      0.83      0.79       336

    accuracy                           0.74      1249
   macro avg       0.75      0.74      0.74      1249
weighted avg       0.76      0.74      0.74      1249
```

Obr. 24: Sieť 2 – Klasifikačný report

```
Model: Model_Large_Filters
Accuracy: 0.7366
Training Time: 362.43 seconds
Confusion Matrix:
[[216   6  99  13]
 [ 12 204  29  54]
 [ 16  22 221  21]
 [  2  26  29 279]]
```

Obr. 25: Sieť 2 - Presnosť, čas trénovania a konfúzna matica v konzole

2.9.3 Experiment 3

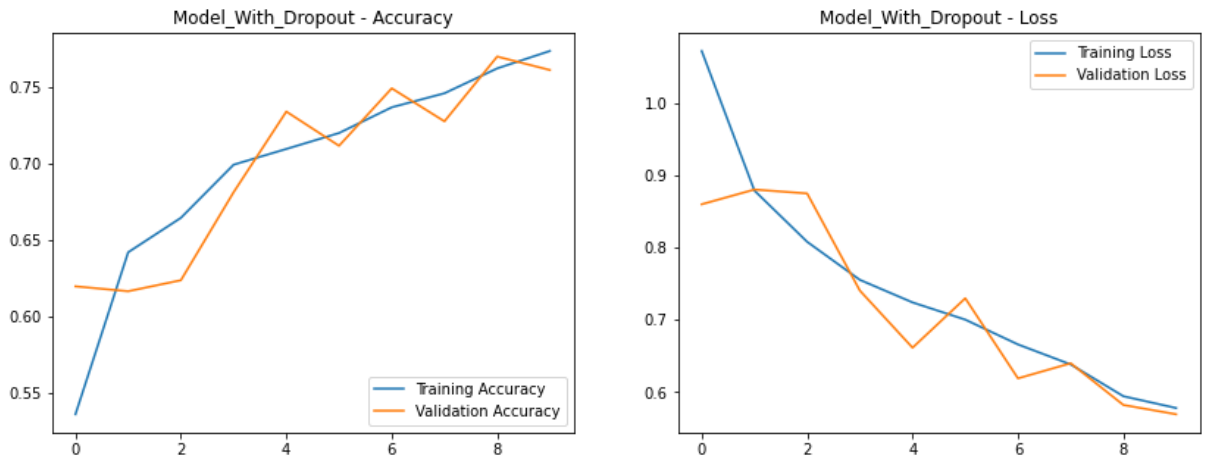
Tu je pridaná dropout vrstva ktorá redukuje overfitting, je tu pridaných viac neurónov v hustej vrstve ktorá vyšuje kapacitu modelu na učenie komplexných vzorov.

```
# Experiment 3: Model with Dropout
results.append(build_and_train_model(filters1=32, filters2=64, dense_units=128, epochs=10, model_name="Model_With_Dropout", use_dropout=True))
```

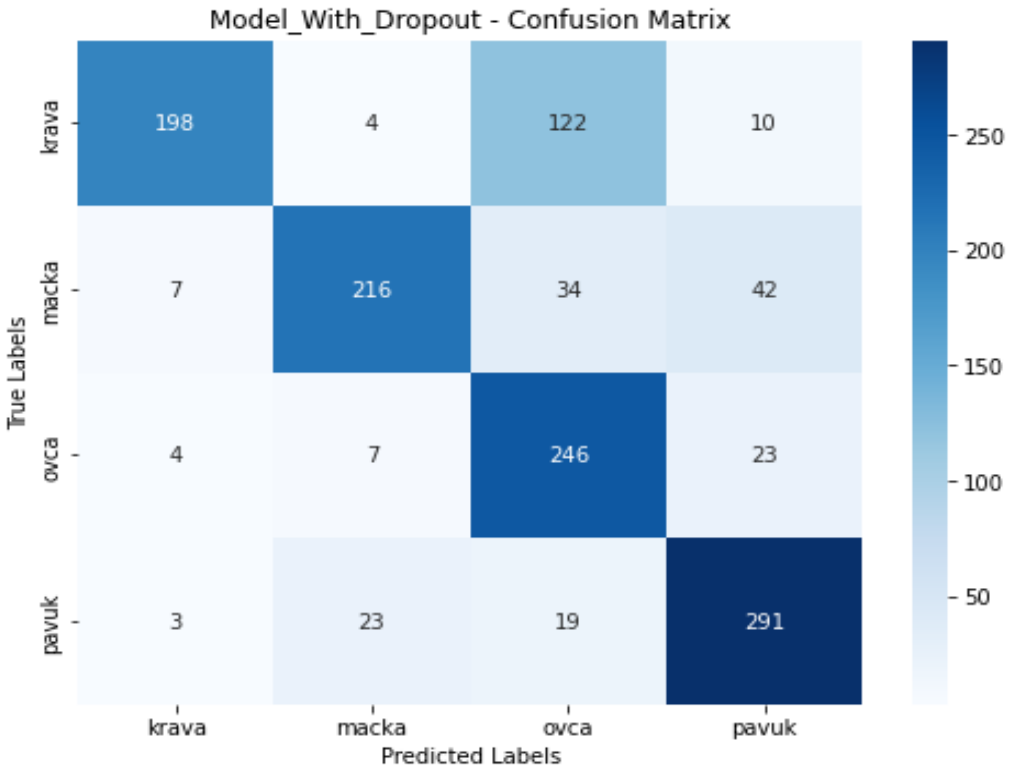
Layer (type)	Output Shape	Param #
conv2d_28 (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d_21 (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_29 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_22 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten_14 (Flatten)	(None, 12544)	0
dense_28 (Dense)	(None, 512)	6,423,040
dropout_1 (Dropout)	(None, 512)	0
dense_29 (Dense)	(None, 4)	2,052

Total params: 6,444,484 (24.58 MB)
Trainable params: 6,444,484 (24.58 MB)
Non-trainable params: 0 (0.00 B)

Obr. 26: Sieť 3 – Architektúra siete



Obr. 27: Sieť 3 – Presnosť a strata



Obr. 28: Sieť 3 – Konfúzna matica

```
Model_With_Dropout - Classification Report:
              precision    recall  f1-score   support

       krava       0.93      0.59      0.73       334
       macka       0.86      0.72      0.79       299
        ovca       0.58      0.88      0.70       280
       pavuk       0.80      0.87      0.83       336

    accuracy                           0.76      1249
   macro avg       0.79      0.76      0.76      1249
weighted avg       0.80      0.76      0.76      1249
```

Obr. 28: Sieť 3 – Klasifikačný report

```
Model: Model_With_Dropout
Accuracy: 0.7614
Training Time: 466.91 seconds
Confusion Matrix:
[[198   4 122  10]
 [  7 216  34  42]
 [  4   7 246  23]
 [  3  23  19 291]]
```

Obr. 29: Sieť 3 – Presnosť, čas trénovania a konfúzna matica v konzole

2.10 Vyhodnotenie experimentov

Po vykonaní troch experimentov sme analyzovali presnosť, čas trénovania a správanie modelov počas trénovania a testovania. Každý experiment priniesol rôzne výsledky, ktoré reflektovali rozdielne nastavenia modelov a ich schopnosť generalizovať dáta.

2.10.1 Experiment 1

Prvý experiment použil malé filtre (16 a 32) s veľkosťou 3x3 a hustú vrstvu s 64 neurónmi. Model dosiahol presnosť **72,78%.** Z grafu trénovania vidíme, že krivka presnosti na tréningových dátach bola pomerne konzistentná s testovacími dátami, čo naznačuje, že model nemá výrazný problém s overfittingom**.** Model dokázal rozlíšiť triedy pomerne dobre, ale niektoré triedy boli zamenené kvôli obmedzenej kapacite menších filtrov. Model je vhodný pre rýchle trénovanie, ale jeho kapacita je obmedzená, čo sa odrazilo na nižšej presnosti.

2.10.2 Experiment 2

Druhý experiment použil väčšie filtre (64 a 128) s veľkosťou 5x5, hustú vrstvu s 256 neurónmi a väčší batch size (64). Tento model dosiahol presnosť **73,66%.** Presnosť tréningu bola vyššia ako v prvom experimente a krivka bola stabilnejšia, čo naznačuje lepšiu schopnosť učenia detailov v dátach. Model dokázal lepšie rozlíšiť triedy s väčším množstvom detailov, ale stále sa vyskytovali drobné chyby, najmä pri menej výrazných triedach. Väčšie filtre a viac neurónov umožnili modelu lepšie zachytávať detaily, čo zvýšilo presnosť na testovacej sade. Tento model bol však náročnejší na výpočty a trénovanie bolo pomalšie.

2.10.3 Experiment 3

Tretí experiment pridal Dropout vrstvu (50%) a hustú vrstvu s 512 neurónmi. Model bol trénovaný na 10 epoch a dosiahol presnosť **76,14%**. Z grafu vidíme, že presnosť na tréningových dátach bola veľmi vysoká, ale testovacia presnosť vykazovala mierne odchýlky. Dropout znížil riziko overfittingu, čo pomohlo dosiahnuť stabilnejšie výsledky. Model rozlíšil triedy veľmi presne a minimalizoval počet zamenených tried. Pridaním Dropout vrstvy a väčšieho počtu neurónov sa model stal robustnejším voči overfittingu a dosiahol najvyššiu presnosť zo všetkých experimentov. Tento model je najvhodnejší pre zložitejšie úlohy.

2.11 Záver

V rámci tohto zadania sme sa venovali vývoju, trénovaniu a analýze konvolučných neurónových sietí (CNN) na klasifikačnej úlohe s cieľom maximalizovať presnosť modelov a porovnať ich výkonnosť v rôznych konfiguráciách. Pri riešení jednotlivých úloh sme postupne implementovali základnú verziu siete, experimentovali s rôznymi parametrami a architektúrami, pridávali Dropout vrstvy a analyzovali výsledky pomocou konfúznej matice, klasifikačných reportov a tréningových grafov. Hlavným prínosom tejto práce bolo pochopenie toho, ako jednotlivé architektonické a tréningové rozhodnutia ovplyvňujú výkonnosť modelu. Pri testovaných konfiguráciách sme dosiahli maximálnu presnosť **76,14%**, čo je blízko maximálnemu potenciálu daného datasetu a architektúry.